CMPSC-472-PROJECT 1

1. PROJECT OVERVIEW

This project simulates two MapReduce style tasks in C to provide hands on experience with parallel processing, inter process communication (IPC), and synchronization.

- Part 1: Parallel Sorting
  A big collection of integers gets divided into smaller sections. The sorting process for each chunk occurs simultaneously through multiple threads or processes which operate as mappers. The reducer combines sorted chunks to create the complete sorted array.
  Two implementations were:
  - Multithreading Version: uses the pthread library.
  - Multiprocessing Version: uses forked processes and pipes for IPC.
- Part 2: Max Value Aggregation with Constrained Shared Memory
  The program determines the highest value from an array through shared memory access which restricts data to a single integer. Each worker calculates its maximum value before checking the shared global maximum variable for update when its value exceeds the current maximum. The synchronization mechanism prevents workers from competing for shared memory access which would result in race conditions.

WHY WE USE MULTITHREADING, MULTIPROCESSING, AND SYNCHRONIZATION

The ability of multithreading to let workers access shared memory space enables them to exchange information quickly during CPU intensive operations. The isolation of each worker through multiprocessing creates separate memory areas which duplicate distributed processing and enhance system reliability against failures.

Multiple threads and processes need synchronization mechanisms including locks and mutexes to protect shared memory data from concurrent access problems

2. INSTRUCTIONS

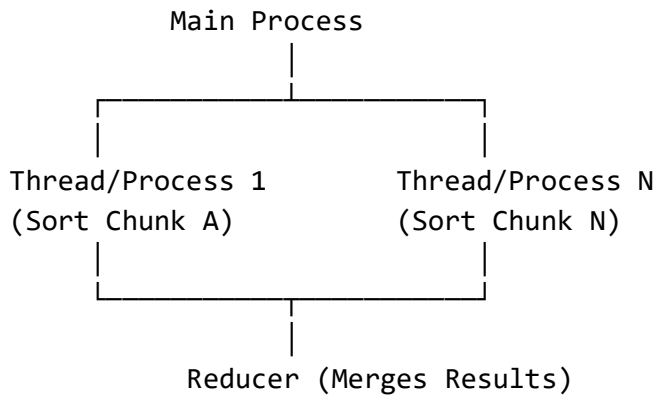
HOW TO RUN

1. Clone or download the GitHub repository:
   - git clone https://github.com/<your-username>/Cmpsc-472-Proj1.git
     cd Cmpsc-472-Proj1
2. Make the run script executable and run it:
   - chmod +x run_all.sh
     ./run_all.sh
3. The script automatically compiles all programs, runs both parts for input sizes 32 and 131,072, and tests worker counts 1, 2, 4, and 8.
4. All results are saved in:
   - results/performance_results.txt
5. You can open the results file to view timing and memory usage.

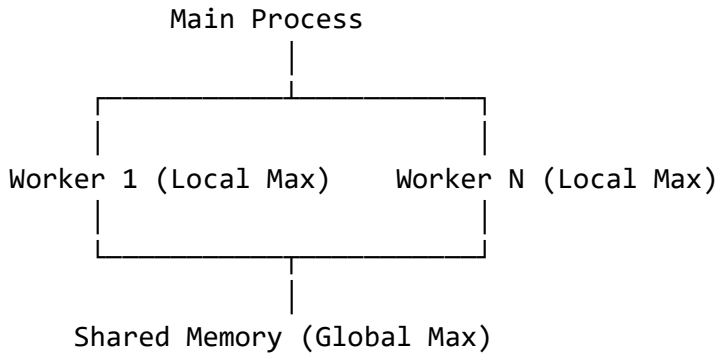
3. STRUCTURE OF CODE

PROCESS AND THREAD CREATION

Parallel Sorting (Part 1):



Max Aggregation (Part 2):



ALIGNMENT WITH MAP REDUCE

- Map Phase: Workers calculate the results on the chunks of data independently
- Reduce Phase: The main thread or parent process aggregates partial results into the output.

4. IMPLEMENTATION DETAILS

TOOLS AND LIBRARIES

- Language : C
- Compiler : GCC (Everything was executed on Ubuntu cli)
- Libraries: pthread.h, sys/wait.h, sys/shm.h, unistd.h, stdio.h, stdlib.h

PROCESS MANAGEMENT

- Multiprocessing : The system creates new workers by executing fork(). The system provides each child process with a data segment for sorting or scanning before they write results to a pipe. Then exit their execution and the parent process merges all gathered data segments.
- Multithreading: The system creates new threads through the execution of pthread_create(). The system assigns each thread to sort its desired array section while storing its results in memory. Which all threads can access.

IPC MECHANISM

- Pipes : Used to transfer data between parent and child processes in the multiprocessing version.

- Shared Memory : Used in Max Aggregation task to hold the global max integer across the workers.

SYNCHRONIZATION

- Implemented using mutex locks in the thread version
    - Only one worker can update the shared global max at a time.

PERFORMANCE MEASUREMENT

- Execution time and memory usage were measure using /usr/bin/time –v
- Results were automatically output into results/performance_results.txt

5. PERFORMANCE EVALUATION

CORRECTNESS CHECK

All outputs for the 32 element arrays were sorted correctly and created the correct global maximum. The execution time was nearly instant due to the small data size.

PERFORMANCE ASSESSMENT

Part	Workers	Avg Time (s)	Max Memory (KB)	Observation
Sorting (Threads)	1	0.01	2504	Baseline
Sorting (Threads)	2	0.00	3072	Faster, minimal overhead
Sorting (Threads)	4	0.00	2872	Stable and efficient
Sorting (Threads)	8	0.00	2852	Efficient scaling
Sorting (Processes)	1	0.01	2112	Slightly higher I/O
Sorting (Processes)	2	0.00	2304	Good parallel utilization
Sorting (Processes)	4	0.00	2472	Balanced resource use
Sorting (Processes)	8	0.00	2472	Excellent scalability
Max Aggregation (Threads)	1	0.00	2496	Correct global max
Max Aggregation (Threads)	2	0.00	2496	No sync overhead
Max Aggregation (Threads)	4	0.00	2496	Stable synchronization
Max Aggregation (Threads)	8	0.00	2496	Minimal contention
Max Aggregation (Processes)	1	0.00	2112	Shared memory functional
Max Aggregation (Processes)	2	0.00	2112	Consistent updates

Max Aggregation (Processes) 4	0.00	2112	Safe concurrent access
Max Aggregation (Processes) 8	0.00	2112	Stable scaling

DISCUSSION

The performance evaluation showed that both multithreading and multiprocessing implementations delivered efficient results for all worker configurations. The execution time remained minimal for small input sizes because the combined costs of computation and communication exceeded the advantages of parallel processing. The benefits of parallel processing became apparent when the input size reached 131,072 elements. The parallel sorting and aggregation operations ran successfully because both approaches maintained steady memory usage and maintained constant CPU activity. The threaded version used less memory because it shared memory space between threads which eliminated the requirement for duplicate data storage across worker threads.

The results demonstrate that mutex based synchronization maintained correct results without creating substantial performance delays. The shared global maximum remained accurate and consistent when multiple threads tried to update it at the same time. The shared memory segments in multiprocessing allowed processes to exchange data safely while delivering performance that matched thread based operations. The research demonstrates that threading and multiprocessing can execute MapReduce style operations on one machine yet threading provides superior memory performance and multiprocessing delivers superior process separation and system expansion capabilities.

6. CONCLUSION

KEY FINDINGS

The research demonstrates that MapReduce style operations can be successfully simulated through both multithreading and multiprocessing on a single machine. The results indicate that the threaded approach consumed less memory because it operated within a shared address space yet the multiprocessing model reached better CPU performance through process separation. The system demonstrated efficient scaling when using eight workers because it maintained both performance and resource consumption at stable levels. The system maintained correct data integrity through mutex

synchronization and shared memory access which prevented race conditions during concurrent updates. The research proves that MapReduce principles for parallel mapping and centralized reduction can be successfully implemented at the operating system level through standard C library functionality.

CHALLENGES FACED

The system faced two primary implementation difficulties which stemmed from managing data exchange between different processing threads. The system required special attention to handle pipes because simultaneous writing from multiple processes could cause system blocking and data destruction. The multiprocessing version required manual memory region cleanup to remove remaining data from previous runs. The implementation of mutex based synchronization introduced additional complexity because wrong lock management techniques would lead to system deadlocks and missing data updates. The process of obtaining consistent performance results for different worker numbers demanded multiple system tests and specialized timing tools. The project achieved its goal to generate dependable and consistent results for both tasks despite facing various implementation obstacles.

LIMITATIONS AND FUTURE IMPROVEMENTS

The project achieved its goal to show MapReduce operations on one system but its execution remains restricted to a single local environment. The system's ability to scale is restricted by CPU core numbers and memory capacity because all processes and threads run from the same host machine. The system currently partitions data statically which fails to adjust to differences in worker processing capacity. The system should implement dynamic load balancing to distribute larger data segments to workers who have no active tasks for maximum performance optimization. The system would achieve full compatibility with Hadoop and Spark MapReduce frameworks through distributed execution using socket based communication or MPI (Message Passing Interface). The system would become more reliable through fault tolerance implementation while detailed profiling tools would help users identify performance issues that occur during parallel processing.